If you think this line looks familiar, it's because you've seen it before. It appears in orders_controller.rb in the Depot application.

Reading Existing Rows

Reading from a database involves first specifying which particular rows of data you're interested in—you'll give Active Record some kind of criteria, and it will return objects containing data from the row(s) matching the criteria.

The most direct way of finding a row in a table is by specifying its primary key. Every model class supports the find method, which takes one or more primary key values. If given just one primary key, it returns an object containing data for the corresponding row (or throws an ActiveRecord::RecordNotFound exception). If given multiple primary key values, find returns an array of the corresponding objects. Note that in this case a RecordNotFound exception is raised if *any* of the IDs can't be found (so if the method returns without raising an error, the length of the resulting array will be equal to the number of IDs passed as parameters).

```
an_order = Order.find(27)   # find the order with id == 27

# Get a list of product ids from a form, then
# find the associated Products
product_list = Product.find(params[:product_ids])
```



David says:

To Raise or Not to Raise?

When you use a finder driven by primary keys, you're looking for a particular record. You expect it to exist. A call to Person.find(5) is based on our knowledge of the people table. We want the row with an ID of 5. If this call is unsuccessful—if the record with the ID of 5 has been destroyed—we're in an exceptional situation. This mandates the raising of an exception, so Rails raises RecordNotFound.

On the other hand, finders that use criteria to search are looking for a *match*. So, Person.where(name: 'Dave').first is the equivalent of telling the database (as a black box)